Zenith: learning from a reasoning graph's mistakes

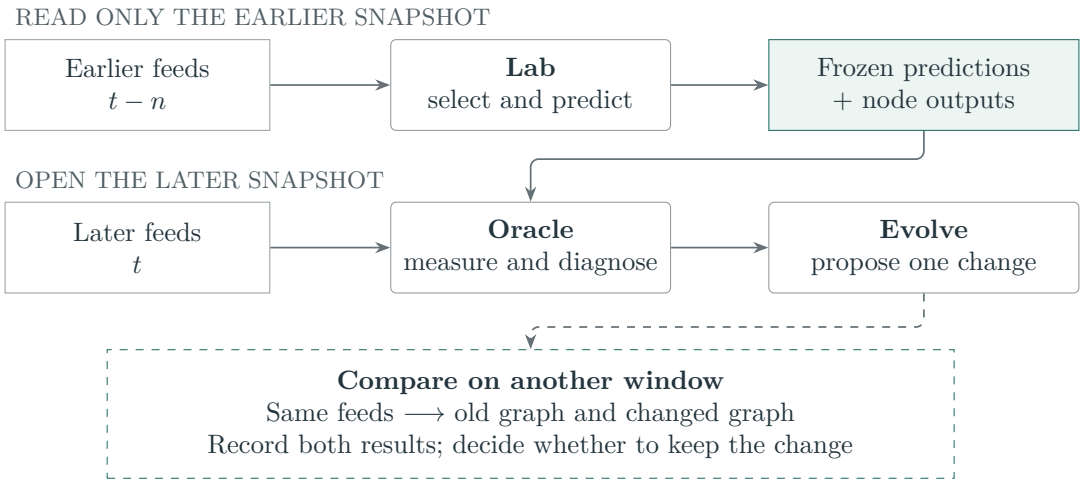
Will Cook • 12 October 2026

This is not financial advice. This paper describes experimental research. Historical examples and model outputs are not investment recommendations and must not be used to make financial decisions.

We have a network of reasoning nodes. Each node is given some evidence and produces an analysis or prediction that later nodes read. Can that procedure learn from its mistakes? The aim is efficient forecasting across market regimes. The example below tests an execution repair, not forecasting skill.

Take the actual saved feeds at an earlier time. Let **Lab** read them, choose the assets worth predicting and save its predictions. Then open the later snapshot. **Oracle** compares the predictions with what happened and proposes an explanation. **Evolve** proposes a change to the procedure. Test the changed and unchanged versions on the same later inputs before deciding what to keep.

The model weights stay fixed. We change a node's instruction, its incoming context or its connections. The hard part is whether the change improves a later prediction, rather than just making an old mistake easier to explain.



The arrows show the experiment's data flow. The dashed step needs a separate comparison; a plausible diagnosis does not complete it.

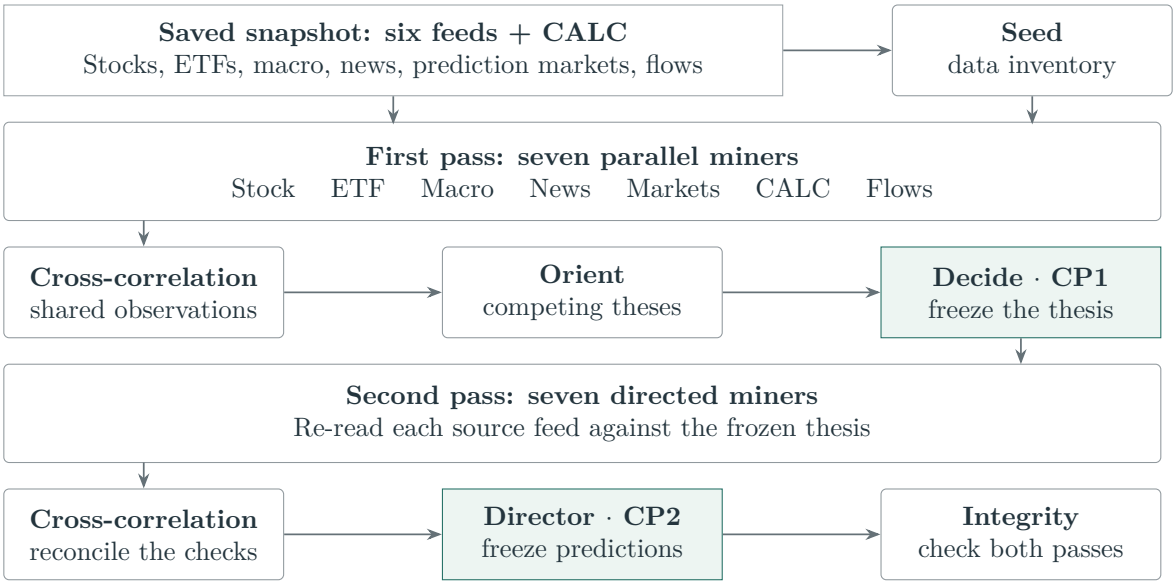
Successive version comparisons draw on AlphaGo Zero [4]; bounded edits followed by evaluation also appear in autoresearch [5]. Here the histories are finite and externally generated.

Start with the data we already have. The local archive contains the original six feeds: stocks, ETFs, macro series, Stockgrid, prediction markets and news. A calculator derives cross-asset summaries from those inputs. For the worked replay we use the saved 20 and 21 July 2026 snapshots [1]. Each contains all seven artifacts. The earlier stock payload has 457 entries; 456 stocks have numeric prices in both snapshots.

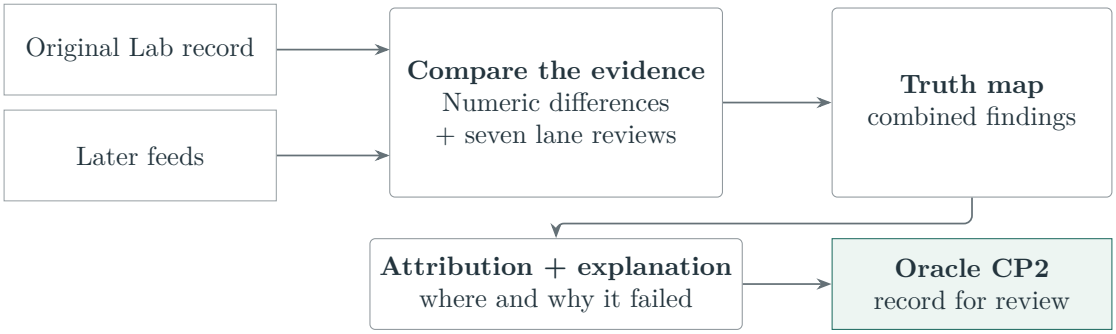
This is a retrospective replay executed now with a current hosted model. Its training data may contain later information. We are testing the operation of the procedure on saved data, not recreating what that model could have known in July or establishing a trading advantage.

A snapshot can contain a history. A point-in-time packet may contain a state (a "photo") or dated histories (a "video" up to the cutoff). Either can target a later state; forecasting a future path needs its own horizons and loss. Future work would use authorised proprietary snapshots with verified availability times and revision vintages. Richer histories do not remove missing information or look-ahead risk.

What happens inside the graph. The Lab has 28 nodes: seven feed artifacts and 21 reasoning operators. Save every output to trace the first unsupported claim.



Each wide miner box contains seven parallel analyses. The second pass re-reads the original feeds against the frozen thesis. In the quantile branch, CP1 also freezes forecast quantiles; Director must preserve them.

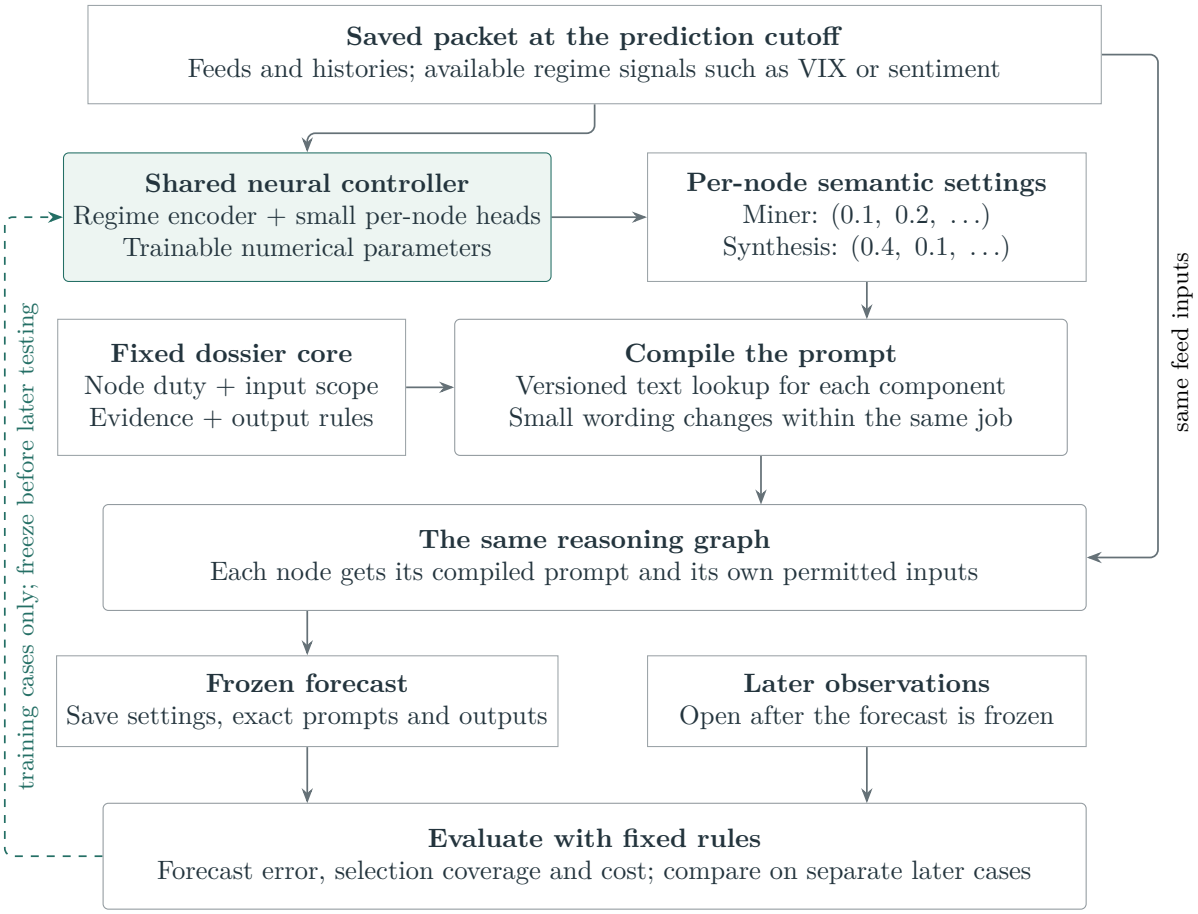


Oracle compares saved arguments with later observations. Attribution proposes where reasoning failed; CP2 records the review without rewriting the forecast. The diagnosis still needs a test.

Figures omit direct second-pass feed inputs, other skip connections and Integrity's cross-pass inputs. Exact dependencies appear in `docs/graphs.md`. Square boxes are records; rounded boxes are operations or named groups.

Trace before changing. Was useful evidence absent, overlooked, grouped badly or misinterpreted? Save the first node where the claim lost support. A repeated error motivates a test; a persuasive explanation does not prove the fix.

Research direction: learn how each node does the same job. A shared neural controller could read market conditions available at the cutoff and choose a few semantic settings for each reasoning node. Each setting in $[0, 1]$ selects a small wording variation in one prompt component. The node's purpose, inputs, work budget and output contract stay fixed. This mechanism is proposed, not implemented or validated by the example.



Solid arrows show data flow. The dashed return fits a future controller from training outcomes; it does not feed a test outcome into its own forecast or denote a derivative through the LLM. The numbers are illustrative settings, not learned values. The shared encoder's internal weights are a different object.

A small semantic change, not a different task. The Polymarket miner still reads seed state and its feed and extracts the same kind of ranked signals. A component at 0.1 might say "consider sentiment as one explanation"; at 0.2, "give sentiment slightly more interpretive weight where supported." Factual measurements, ranking rules and required checks remain fixed. Other nodes have their own bounded interpretive components. Neither a high setting nor a high market indicator makes a claim true.

What can be trained. Search discrete settings, or train the numerical controller from measured losses. Ordinary backpropagation requires local derivatives (see micrograd); hard text lookup and hosted LLM calls expose no such path. DSPy provides a prompt-search reference. The base LLM stays fixed. VIX and sentiment are incomplete regime features; robustness to every situation is not assured. Detailed proposal and source references: `docs/research-direction.md`.

A measured failure and one change. Four full Lab attempts used original snapshots from 26 June, 13 July, 20 July and 21 July 2026. All used DeepSeek V4.1 Flash through DeepInfra, the same graph and policy, complete feed data, and a 4,096-token response limit. Each completed the inventory and seven miners, then stopped when Cross-correlation's JSON response ran out of tokens [2].

We added one instruction at that node: keep the four required arrays, at most three records per array, at most four subjects per record, and short summaries. We froze the instruction after inspecting the two earlier episodes. On the two later inputs, the changed node received exactly the baseline miners' saved outputs and went through the native validator:

Later input	Original instruction	Changed instruction
20 July close	truncated at 4,096	valid JSON, 1,308 tokens
21 July open capture	truncated at 4,096	valid JSON, 1,172 tokens

Both changed responses still exceeded the requested four-subject bound: their largest records had nine and eleven subjects. The native schema permits this. The measured gain is local execution validity, with incomplete instruction compliance. It says nothing yet about prediction quality.



A separate conditional continuation reused the eight unchanged July 20 ancestor outputs and seven feeds. It made three new calls with the frozen change. Cross-correlation and Orient passed; Decide failed. No completed CP1, CP2 or forecast resulted [3].

What to measure next. Complete Lab first. Then measure selection and forecast error separately: which assets were selected, coverage, their frozen predictions and later observations. When scoring the whole universe, fix the same fallback for unselected assets. Use a proper score such as quantile loss for quantile forecasts [6]; abstention must not become a zero-loss win.

The later windows overlap, the analyst had already seen the July 20 failure, and there is one model sample per arm. These are not blind, independent trials. Collection times establish availability of the saved files, not the clocks of their underlying price bars. Until those clocks are verified, price changes are differences between saved values, not a clean financial backtest.

An agent authored and tested this execution repair. The full Oracle/Evolve loop has not run, and improved forecasting remains untested.

References

[1] Zenith. Saved original feed runs `FEEDS_20260720_close` and `FEEDS_20260721_close`. Source hashes and example rows: `examples/replay/sources.json` and `observations.json`.

[2] Zenith. Original-feed window experiment, 12 October 2026. `docs/original-feed-window-result.json` and `original-feed-windows.md`.

[3] Zenith. Conditional native Lab continuation, 12 October 2026. `examples/replay/conditional-result.json`.

[4] D. Silver and D. Hassabis. *AlphaGo Zero: Starting from scratch*. DeepMind, 2017. https://deepmind.google/blog/alphago-zero-starting-from-scratch/.

[5] A. Karpathy. *autoresearch*. Source inspected at `228791fb499a`. https://github.com/karpathy/autoresearch.

[6] T. Gneiting and A. E. Raftery. Strictly proper scoring rules, prediction, and estimation. *JASA* 102(477):359–378, 2007. https://doi.org/10.1198/016214506000001437.